

基于中间语言与指令模板的多语言 DFA 构建及鲲鹏环境验证

计算机 2303 尤颢辰

April 2026

1 设计正则语言簇 $\mathcal{L}$

1.1 QTAC 是什么？

QTAC 就是一种中间语言，是编译器在”源代码”和”机器码”之间的一个过渡层。它比 C 语言更底层（更接近机器），但比 ARM 汇编更高层（更容易理解）。它的特点是：每条指令最多做一件事、用 LABEL 和 GOTO 代替 while/for、用临时变量拆解复杂表达式。该报告完成的任务，就是编写一个编译器后端：把 QTAC 中间代码翻译成 ARM64 汇编。QTAC 格式简单、规整，每条指令结构固定，所以很适合用 DFA 来逐条识别和翻译。

1.2 定义输入字母表 Σ

对 QTAC 有一定的了解之后，我们就可以按照构建 DFA 的思路，来一步步完成编译器的构建了。我们把 QTAC 代码做词法分析后，每个 token 归为一个符号类别。对于 Fact 程序用到的部分， Σ 如下：

符号	含义	举例
LABEL(操作码)	关键字 LABEL	LABEL
IF(操作码)	关键字 IF	IF
THEN(操作码)	关键字 THEN	THEN
ELSE(操作码)	关键字 ELSE	ELSE
GOTO(操作码)	关键字 GOTO	GOTO
RETURN(操作码)	关键字 RETURN	RETURN
q(操作数类型)	变量或临时变量	n, a, t1, t2
k(操作数类型)	立即数	2, 1
l(操作数类型)	标号名	l0, l1, l2
=(操作码)	赋值号	=
aop(操作数类型)	算术运算符	+, -, *, /

接下页...

符号	含义 (续)	举例 (续)
rop(操作数类型)	关系运算符	<, >, <=, >=, ==, !=
;(操作码)	分号 (语句结束)	;

这样，我们就定义好了这个多语言联合 DFA 的字母表。

1.3 选取模板

分析完毕字母表所有可能的成员之后，我们就可以分析 Fact 的 QTAC 代码用到了哪些指令类型了。随后，再从模板库里挑出对应的模板，就可以进行 $\sigma - DFA$ 的构建了。

Fact QTAC 代码中出现的指令类型如下表所示：

指令形式	出现在 Fact 中的具体指令	对应选取的模板
LABEL l;	LABEL l0; 等	T_LABEL
q = k;	t1 = 2;	T_MOV
q = q;	n = t2;、a = t3;	T_MOV
q = q aop k;	t2 = n - 1;	T_AOP
q = q aop q;	t3 = n * a;	T_AOP
IF q rop q THEN l ELSE l;	IF n < t1 THEN l1 ELSE l2;	T_IF / T_IF_LABEL
GOTO l;	GOTO l0;、GOTO l3;	T_GOTO
RETURN q;	RETURN a;	T_RETURN

从 Q2ARM 模板库中选取以下 7 个模板。其模板名和功能如下表所示：

模板	功能
T_LABEL	标号定义
T_MOV	赋值（寄存器间或立即数赋值）
T_AOP	算术运算
T_IF_LABEL	条件跳转 + 紧跟标号（优化模式）
T_IF	条件跳转（后面不紧跟标号时的备用）
T_GOTO	无条件跳转
T_RETURN	函数返回

1.4 将模板转化为正则表达式

对照 Q2ARM 模板，我们为 Fact 程序选用的 7 个模板编写正则表达式：

正则语言	正则表达式
$L_1(\text{T_LABEL})$	<code>LABEL l ;</code>
$L_2(\text{T_MOV})$	<code>q = (q k) ;</code>
$L_3(\text{T_AOP})$	<code>q = q aop (q k) ;</code>
$L_4(\text{T_GOTO})$	<code>GOTO l ;</code>
$L_5(\text{T_RETURN})$	<code>RETURN (q k) ;</code>
$L_6(\text{T_IF_LABEL})$	<code>IF q rop (q k) THEN l ELSE l ; LABEL l ;</code>
$L_7(\text{T_IF})$	<code>IF q rop (q k) THEN l ELSE l ;</code>

其中， L_6 和 L_7 存在前缀重叠，我们采用最长匹配原则区分；其余语言首符号互不相同，两两不相交。经过上面的一系列工作，我们就设计好了这个编译器的目标正则语言簇：

$$\mathcal{L} = \{L_1, L_2, L_3, L_4, L_5, L_6, L_7\}$$

2 构建 $\sigma - DFA$

2.1 定义状态集合 Q

我们现在开始构建这个多语言联合 DFA。首先，我们来定义状态集合 Q 。在这个步骤，我们列出 DFA 的所有状态（初始状态、中间状态、接受状态）。状态如下所示：

初始状态： q_0

接受状态： $ACC1, ACC2, ACC3, ACC4, ACC5, ACC6, ACC7$ 。这七个接受状态分别匹配正则语言簇里的七种正则语言。

中间状态： $q_1 \rightarrow q_{23}$ ，这些是匹配过程中的中间状态。

2.2 设计状态转移函数

由于数量很多，写一份状态转移表出来不实际，而且状态转移表很多位置都是空。所以下面用状态转移图的方式来说明具体转移函数的实现：

第一条路径：通往 T_LABEL 接受状态：

$$q_0 \xrightarrow{\text{LABEL}} q_1 \xrightarrow{l} q_2 \xrightarrow{i} ACC_1$$

第二条路径：通往 T_MOV 与 T_AOP 接受状态：

$$q_0 \xrightarrow{q} q_3 \xrightarrow{=} q_4 \xrightarrow{k} q_5 \xrightarrow{i} ACC_2 \quad (\text{T_MOV})$$

$$q_4 \xrightarrow{q} q_6 \xrightarrow{i} \text{ACC}_2 \quad (\text{T_MOV})$$

$$q_6 \xrightarrow{\text{aop}} q_7 \xrightarrow{q|k} q_8 \xrightarrow{i} \text{ACC}_3 \quad (\text{T_AOP})$$

第三条路径：通往 T_GOTO 接受状态：

$$q_0 \xrightarrow{\text{GOTO}} q_9 \xrightarrow{1} q_{10} \xrightarrow{i} \text{ACC}_4$$

第四条路径：通往 T_RETURN 接受状态：

$$q_0 \xrightarrow{\text{RETURN}} q_{11} \xrightarrow{q|k} q_{12} \xrightarrow{i} \text{ACC}_5$$

第五条路径：通往 T_IF 与 T_IF_LABEL 接受状态：

$$q_0 \xrightarrow{\text{IF}} q_{13} \xrightarrow{q} q_{14} \xrightarrow{\text{rop}} q_{15} \xrightarrow{q|k} q_{16} \xrightarrow{\text{THEN}} q_{17}$$

$$q_{17} \xrightarrow{1} q_{18} \xrightarrow{\text{ELSE}} q_{19} \xrightarrow{1} q_{20} \xrightarrow{i} q_{21}$$

$$q_{21} \xrightarrow{\text{LABEL}} q_{22} \xrightarrow{1} q_{23} \xrightarrow{i} \text{ACC}_7 \quad (\text{T_IF_LABEL})$$

$$q_{21} \xrightarrow{\text{非 LABEL}} \text{ACC}_6 \quad (\text{T_IF}), \text{当前符号退回重新匹配}$$

另，对于所有的状态，如果在上面的路径中没有找到合法的转移，默认进入陷阱状态（突然死亡）。

2.3 附加信息的识别与存储

单独的状态转移函数 δ 只能识别 QTAC 指令的结构类型，无法还原具体内容（例如变量名、立即数值、标号名、运算符等）。因此需要在状态转移过程中，额外维护一个参数缓冲区 P，记录构成模板参数所需的附加信息。每次从初始状态 q_0 出发启动新一轮匹配时清空 P；到达接受状态时，结合接受状态标签和 P 的内容生成对应的 ARM 指令。

下面的表列出了参数缓冲区所有可能的字段：

字段	含义	来源（读到的符号）
dest	目标变量	$q_0 \rightarrow q_3$ 读到 q
src1	第一源操作数（变量）	$q_4 \rightarrow q_6$ 读到 q
src2	第二源操作数（变量或立即数）	$q_4 \rightarrow q_5$ 读到 k，或 $q_7 \rightarrow q_8$ 读到 q/k
src2_type	第二源操作数类型（q 或 k）	同上，根据读到的符号类别记录
op	算术运算符	$q_6 \rightarrow q_7$ 读到 aop

接下页...

字段 (续)	含义 (续)	来源 (读到的符号) (续)
lhs	IF 左操作数	$q_{13} \rightarrow q_{14}$ 读到 q
rhs	IF 右操作数 (变量或立即数)	$q_{15} \rightarrow q_{16}$ 读到 q/k
rop	关系运算符	$q_{14} \rightarrow q_{15}$ 读到 rop
label_true	IF 真分支标号	$q_{17} \rightarrow q_{18}$ 读到 l
label_false	IF 假分支标号	$q_{19} \rightarrow q_{20}$ 读到 l
label_next	IF 后紧跟的标号	$q_{22} \rightarrow q_{23}$ 读到 l
target	GOTO 跳转目标	$q_9 \rightarrow q_{10}$ 读到 l
ret_val	返回值	$q_{11} \rightarrow q_{12}$ 读到 q/k
label_name	定义的标号名	$q_1 \rightarrow q_2$ 读到 l

在筛选出所有可能的字段后,我们就可以再列出一个表格,统计每一个匹配到的模板需要使用哪些字段以及最后生成的 ARM 指令形式了。表格如下:

接受状态	匹配模板	缓冲区使用字段	生成的 ARM 指令
ACC1	T_LABEL	label_name	{label_name}:
ACC2	T_MOV	dest, src2, src2_type	MOV X{dest}, X{src2} 或 MOV X{dest}, #{src2}
ACC3	T_AOP	dest, src1, op, src2, src2_type	{Xaop/Kaop} X{dest}, X{src1}, X{src2}/#{src2}
ACC4	T_GOTO	target	B {target}
ACC5	T_RETURN	ret_val, ret_type	MOV X0, X{ret_val}; RET 或 MOV X0, #{ret_val}; RET
ACC6	T_IF	lhs, rop, rhs, label_true, label_false	CMP X{lhs}, X{rhs}; B.{Jrop} {label_true}; B {label_false}
ACC7	T_IF_LABEL	ACC 6 的字段 + label_next	根据 label_next 是否等于 label_false 选择优化形式

2.4 该部分的其他阐述

关于 DELTA 转移表的设计,根据我们上面的设计,行数 $|Q| = 31$,列数 $|\Sigma| = 13$ 。由于里面绝大部分都是空的 (进入陷阱状态),完整列出意义不大。在上面我给出的状态转移路径里列出了所有有效路径,如何转移已经很明了了。

关于附加信息的管理方式,我采用独立的参数缓冲区 P 来管理附加信息。DELTA 只负责状态转移, P 负

责记录模板参数。两者通过”附加动作”关联：每次转移时，转移动作指定是否向 P 写入数据，以及写入哪个字段。附加信息有两个用处：1. 个别状态的转移需要根据附加信息做决策。本设计中只有一处： q_21 判断 label_next 是否与 label_false 相等，从而选择生成优化形式的 T_IF_LABEL 指令。2. 用于生成 ARM 指令：绝大多数附加信息属于此类——DFA 识别阶段保存，后处理阶段使用。

关乎紧耦合的解耦处理，我按照指导书的建议使用了两遍处理。第一遍输出记号串，第二遍（当作词法分析框架中的后处理）将词法记号对照 ARM 指令模式逐一转换输出。这样能良好地实现我们的要求。首先 DFA 只输出模板记号串，不涉及 ARM 指令；其次识别和生成分离，避免了逻辑混乱；最后修改 ARM 规则只需改后处理器，不涉及 DFA 的修改。

3 编程实现

下面，我们按照之前设计的 $\sigma - DFA$ ，编写代码生成器。

3.1 实现方式选择

对于实验指导书给出的三种实现方式，我选择了表驱动。理由如下：

1. 步骤 2 中定义的 DELTA 转移表可以直接作为程序的数据结构，状态转移逻辑等价于对表的查询操作；
2. 若后续增加新的指令或 Q2ARM 模板，只需修改 DELTA 和模板规则表，无需改动核心代码；
3. 表驱动天然地将”数据（转移表、模板规则）”和”逻辑（识别器、生成器）”分离，符合解耦原则。

在我的实现里，一共需要维护下面六张表：1.DELTA 转移表，用于模拟转移函数逻辑。2.WRITE_ACTION 写入动作表，用于指定转移的时候向缓冲区写入哪个字段。3.ACCEPT_LABEL 接受状态标签表，用于将接受类型映射为模板类型。4.REG_MAP 寄存器映射表，用于将虚拟寄存器映射为物理寄存器。5.AOP_MAP 算术运算符映射表，用于将 QTAC 的运算符映射为 ARM 指令的运算符（ADD 等）。6.ROP_MAP 关系运算符映射表，用于将 QTAC 关系运算符映射为 ARM 跳转指令（B 等）。由于篇幅原因，这些表的具体内容就不展示出来了。

还有，程序运行中需要有一个缓冲区 P，来存储单条指令匹配过程中的附加信息；还要存下来所有的模板记号串，作为第一遍 DFA 识别的输出和第二遍 ARM 代码生成的输入。

3.2 程序整体架构

代码生成器采用两遍处理架构，整体流程如下：

输入：QTAC 源文件 (fact.qtac)；

词法分析器：将源文本切分为 token，保留字面原始值（如立即数的值），为每个 token 标注类别；

第一遍：DFA 识别器，按照转移函数进行转移，维护缓冲区 P，每到达接受状态输出一个模板记号串；

第二遍：ARM 代码生成器，遍历模板记号串，按模板规则表展开每个模板记号；

输出包装：输出完整的 ARM64 汇编文件 (fact_asm.s)。

3.3 代码实现

在这一阶段，我们要实现的代码有：codegen.py（主要程序，用于将 QTAC 中间代码转换为 ARM 代码），fact.qtac（阶乘函数的 QTAC 中间代码，作为 codegen.py 的输入），main.c（测试主程序，C 语言实现的启

动器，用于调用 fact 函数来测试)。

下面给出这些程序的源码：

fact.qtac:

```
1 LABEL 10;
2 t1 = 2;
3 IF n < t1 THEN l1 ELSE l2;
4 LABEL 11;
5 RETURN a;
6 GOTO l3;
7 LABEL 12;
8 t2 = n - 1;
9 t3 = n * a;
10 n = t2;
11 a = t3;
12 GOTO 10;
13 LABEL 13;
```

main.c

```
1 #include <stdio.h>
2
3
4 extern long fact(long n, long a);
5
6 int main(void) {
7     long n = 5;
8     long a = 1;
9     long result = fact(n, a);
10    printf("fact(%ld, %ld) = %ld\n", n, a, result);
11
12    // 验证
13    if (result == 120) {
14        printf("OK: 5! = 120\n");
15    } else {
16        printf("FAIL: expected 120, got %ld\n", result);
17        return 1;
18    }
19    return 0;
20 }
```

codegen.py:

```
1 import sys
2 from typing import List, Tuple, Dict, Union
3
4 # DFA 状态转移表
5 DELTA = {
6     ("q0", "LABEL"): "q1",
7     ("q0", "q"): "q3",
8     ("q0", "GOTO"): "q9",
9     ("q0", "RETURN"): "q11",
```

```

10     ("q0", "IF"):      "q13",
11     ("q1", "l"):      "q2",
12     ("q2", ";"):      "ACC1",
13     ("q3", "="):     "q4",
14     ("q4", "k"):      "q5",
15     ("q4", "q"):      "q6",
16     ("q5", ";"):      "ACC2",
17     ("q6", ";"):      "ACC2",
18     ("q6", "aop"):    "q7",
19     ("q7", "q"):      "q8",
20     ("q7", "k"):      "q8",
21     ("q8", ";"):      "ACC3",
22     ("q9", "l"):      "q10",
23     ("q10", ";"):     "ACC4",
24     ("q11", "q"):     "q12",
25     ("q11", "k"):     "q12",
26     ("q12", ";"):     "ACC5",
27     ("q13", "q"):     "q14",
28     ("q14", "rop"):   "q15",
29     ("q15", "q"):     "q16",
30     ("q15", "k"):     "q16",
31     ("q16", "THEN"):  "q17",
32     ("q17", "l"):     "q18",
33     ("q18", "ELSE"):  "q19",
34     ("q19", "l"):     "q20",
35     ("q20", ";"):     "q21",
36     ("q21", "LABEL"): "q22",
37     ("q22", "l"):     "q23",
38     ("q23", ";"):     "ACC7",
39 }
40
41 # 附加动作表：转移时向参数缓冲区 P 写入的字段
42 WRITE_ACTION = {
43     ("q0", "q"):      "dest",
44     ("q1", "l"):      "label_name",
45     ("q4", "q"):      ("src1", "q"),
46     ("q4", "k"):      ("src1", "k"),
47     ("q6", "aop"):    "op",
48     ("q7", "q"):      ("src2", "q"),
49     ("q7", "k"):      ("src2", "k"),
50     ("q9", "l"):      "target",
51     ("q11", "q"):     ("ret_val", "q"),
52     ("q11", "k"):     ("ret_val", "k"),
53     ("q13", "q"):     "lhs",
54     ("q14", "rop"):   "rop",
55     ("q15", "q"):     ("rhs", "q"),
56     ("q15", "k"):     ("rhs", "k"),
57     ("q17", "l"):     "label_true",
58     ("q19", "l"):     "label_false",
59     ("q22", "l"):     "label_next",
60 }
61
62 # 接受状态 → 模板类型
63 ACCEPT_LABEL = {
64     "ACC1": "T_LABEL",

```



```

65     "ACC2": "T_MOV",
66     "ACC3": "T_AOP",
67     "ACC4": "T_GOTO",
68     "ACC5": "T_RETURN",
69     "ACC6": "T_IF",
70     "ACC7": "T_IF_LABEL",
71 }
72
73 # 寄存器映射表
74 REG_MAP = {
75     "n": "X19",
76     "a": "X20",
77     "t1": "X1",
78     "t2": "X2",
79     "t3": "X3",
80 }
81
82 # 算术运算符映射
83 AOP_MAP = {"+": "ADD", "-": "SUB", "*": "MUL", "/": "SDIV"}
84
85 # 关系运算符映射
86 ROP_MAP = {"<": "B.LT", ">": "B.GT", "<=": "B.LE", ">=": "B.GE", "==" : "B.EQ", "!=": "B.NE"}
87
88 # 反关系运算符映射 (T_IF_LABEL 优化用)
89 ROP_INV_MAP = {"<": "B.GE", ">": "B.LE", "<=": "B.GT", ">=": "B.LT", "==" : "B.NE", "!=": "B.EQ"}
90
91 # 词法分析: QIAC 源文本 → token 列表
92 def tokenize(source):
93     for sym in ["<=", ">=", "==", "!="]:
94         source = source.replace(sym, f" {sym} ")
95     for sym in [";", "=", "<", ">", "+", "-", "*", "/"]:
96         source = source.replace(sym, f" {sym} ")
97     source = source.replace("< =", "<=")
98     source = source.replace("> =", ">=")
99     source = source.replace("= =", "==")
100    source = source.replace("! =", "!=")
101    return [classify(s) for s in source.split()]
102
103 # 单个字符串 → (符号类别, 字面值)
104 def classify(s):
105     if s in ("LABEL", "IF", "THEN", "ELSE", "GOTO", "RETURN"):
106         return (s, s)
107     if s == "=" or s == ";":
108         return (s, s)
109     if s in ("+", "-", "*", "/"):
110         return ("aop", s)
111     if s in ("<", ">", "<=", ">=", "==", "!="):
112         return ("rop", s)
113     if s.startswith("l") and s[1:].isdigit():
114         return ("l", s)
115     if s.lstrip("-").isdigit():
116         return ("k", int(s))
117     return ("q", s)
118
119 # 第一遍: DFA 驱动识别, 输出模板记号串

```

```

120 def recognize(tokens):
121     output = []
122     state = "q0"
123     P = {}
124     i = 0
125     while i < len(tokens):
126         sym, literal = tokens[i]
127         # q21 最长匹配: 非 LABEL 则接受 T_IF, 当前 token 退回
128         if state == "q21" and sym != "LABEL":
129             output.append(("T_IF", dict(P)))
130             state = "q0"
131             P = {}
132             continue
133         key = (state, sym)
134         if key not in DELTA:
135             raise SyntaxError(f"非法转移: state={state}, symbol={sym}, literal={literal}")
136         next_state = DELTA[key]
137         if key in WRITE_ACTION:
138             action = WRITE_ACTION[key]
139             if isinstance(action, tuple):
140                 field, type_tag = action
141                 P[field] = literal
142                 P[field + "_type"] = type_tag
143             else:
144                 P[action] = literal
145         state = next_state
146         i += 1
147         # 到达接受状态, 输出模板记号并重置
148         if state in ACCEPT_LABEL:
149             output.append((ACCEPT_LABEL[state], dict(P)))
150             state = "q0"
151             P = {}
152         if state == "q21":
153             output.append(("T_IF", dict(P)))
154     return output
155
156 # 查寄存器映射表
157 def reg(name):
158     return REG_MAP[name]
159
160 # 以下为各模板的 ARM 指令展开函数
161 def emit_label(P):
162     return [f"{P['label_name']}:" ]
163
164 def emit_mov(P):
165     dest = reg(P['dest'])
166     if P['src1_type'] == 'q':
167         return [f"    MOV {dest}, {reg(P['src1'])}"]
168     return [f"    MOV {dest}, #{P['src1']}"]
169
170 def emit_aop(P):
171     dest, src1, arm_op = reg(P['dest']), reg(P['src1']), AOP_MAP[P['op']]
172     if P['src2_type'] == 'q':
173         return [f"    {arm_op} {dest}, {src1}, {reg(P['src2'])}"]
174     return [f"    {arm_op} {dest}, {src1}, #{P['src2']}"]

```

```

175
176 def emit_goto(P):
177     return [f"    B {P['target']}"]
178
179 def emit_return(P):
180     if P['ret_val_type'] == 'q':
181         return [f"    MOV X0, {reg(P['ret_val'])}", "    RET"]
182     return [f"    MOV X0, #{P['ret_val']}", "    RET"]
183
184 def emit_if(P):
185     lhs = reg(P['lhs'])
186     rhs = reg(P['rhs']) if P['rhs_type'] == 'q' else f"#{P['rhs']}"
187     return [f"    CMP {lhs}, {rhs}", f"    {ROP_MAP[P['rop']] {P['label_true']}", f"    B {P['label_false']}"
188 ]
189
190 def emit_if_label(P):
191     lhs = reg(P['lhs'])
192     rhs = reg(P['rhs']) if P['rhs_type'] == 'q' else f"#{P['rhs']}"
193     if P['label_next'] == P['label_false']:
194         return [f"    CMP {lhs}, {rhs}", f"    {ROP_MAP[P['rop']] {P['label_true']}", f"{P['label_false']}:"
195 ]
196     return [f"    CMP {lhs}, {rhs}", f"    {ROP_INV_MAP[P['rop']] {P['label_false']}", f"{P['label_true']}:"
197 ]
198
199 # 模板展开调度表
200 EMIT_DISPATCH = {
201     "T_LABEL": emit_label, "T_MOV": emit_mov, "T_AOP": emit_aop,
202     "T_GOTO": emit_goto, "T_RETURN": emit_return, "T_IF": emit_if, "T_IF_LABEL": emit_if_label,
203 }
204
205 # 第二遍：遍历模板记号串，展开为 ARM 汇编
206 def generate(template_tokens):
207     lines = []
208     for template, P in template_tokens:
209         lines += EMIT_DISPATCH[template](P)
210     return lines
211
212 # 输出包装：添加函数头和参数搬运
213 def wrap_output(body_lines):
214     header = [".text", ".global fact", ".type fact, %function", "fact:"]
215     prologue = ["    MOV X19, X0", "    MOV X20, X1"] # n→X19, a→X20
216     return "\n".join(header + prologue + body_lines) + "\n"
217
218 def main():
219     source = sys.stdin.read()
220     tokens = tokenize(source)
221     templates = recognize(tokens)
222     asm_lines = generate(templates)
223     sys.stdout.write(wrap_output(asm_lines))
224
225 if __name__ == "__main__":
226     main()

```

4 在开发板上测试

4.1 测试环境

本次实验，我采用香橙派 OrangePi AiPro (8T) 来测试 ARM 指令的运行情况。下面的命令分别展示开发板的 CPU 架构、操作系统版本、内核版本、CPU 型号和核心数、GCC 版本、Python 版本。

```
OrangePi
Welcome to Orange Pi Ai Pro
This system is based on Ubuntu 22.04.3 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

Last login: Sat Apr 18 16:15:12 2026 from 192.168.137.1
\](base) HwHiAiUser@orangepiaipro:~$ \uname -m
Command 'luname' not found, did you mean:
  command 'uname' from deb uucp (1.07-27build3)
  command 'uname' from deb coreutils (8.32-4.1ubuntu1.2)
Try: apt install <deb name>
(base) HwHiAiUser@orangepiaipro:~$ uname -m
aarch64
(base) HwHiAiUser@orangepiaipro:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.3 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
(base) HwHiAiUser@orangepiaipro:~$ uname -r
5.10.0+
(base) HwHiAiUser@orangepiaipro:~$ lscpu | head -20
Architecture:      aarch64
CPU op-mode(s):    64-bit
Byte Order:        Little Endian
CPU(s):            4
On-line CPU(s) list: 0-3
Vendor ID:         0x00
Model:             2
Thread(s) per core: 1
Core(s) per cluster: 4
Socket(s):         -
Cluster(s):        1
Stepping:          0x1
BogoMIPS:          96.00
Flags:             fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcma dcpop sha3 asimddp sha512 sve asimdghm ssbs sb
NUMA node(s):      3
NUMA node0 CPU(s): 0-3
NUMA node1 CPU(s):
NUMA node2 CPU(s):
Vulnerability Itlb multihit: Not affected
Vulnerability Itlb: Not affected
(base) HwHiAiUser@orangepiaipro:~$ gcc --version | head -1
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
(base) HwHiAiUser@orangepiaipro:~$ python3 --version
Python 3.9.2
```

图 1: Environment

4.2 测试流程

首先，在开发板上运行代码生成器，将 QTAC 源码转换为 ARM64 汇编。命令为 `python3 codegen.py < fact.qtac > fact_asm.s`。

生成的 ARM64 汇编代码如下：

```

Last login: Sat Apr 18 17:17:48 2026 from 192.168.137.1
(base) HwHiAiUser@orangepiaipro:~$ python3 codegen.py < fact.qtac > fact\_asm.s.
-bash: fact.qtac: No such file or directory
(base) HwHiAiUser@orangepiaipro:~$ cd ~/fact_compiler
(base) HwHiAiUser@orangepiaipro:~/fact_compiler$ python3 codegen.py < fact.qtac > fact\_asm.s.
(base) HwHiAiUser@orangepiaipro:~/fact_compiler$ cat fact_asm.s
    .text
    .global fact
    .type fact, %function
fact:
    MOV X19, X0
    MOV X20, X1
l0:
    MOV X1, #2
    CMP X19, X1
    B.GE l2
l1:
    MOV X0, X20
    RET
    B l3
l2:
    SUB X2, X19, #1
    MUL X3, X19, X20
    MOV X19, X2
    MOV X20, X3
    B l0
l3:

```

图 2: ARM64 code

之后,使用 GCC 将 C 驱动程序与生成的汇编文件编译链接为可执行程序。命令为 `gcc -g main.c fact\_asm.s -o fact\_run`。编译过程无错误、无警告,说明生成的汇编语法正确。

最后,执行生成的可执行程序。命令 `./fact\_run`。观察运行输出:

```

(base) HwHiAiUser@orangepiaipro:~/fact_compiler$ gcc -g main.c fact_asm.s -o fact_run
(base) HwHiAiUser@orangepiaipro:~/fact_compiler$ ./fact_run
fact(5, 1) = 120
OK: 5! = 120

```

图 3: Output

计算结果 $120 = 5!$ 与预期一致,验证了代码生成器的正确性。

4.3 踪迹断点检查

下面使用 gdb 进行断点测试,测试几个关键寄存器的值。

```

Breakpoint 1, fact () at fact_asm.s:5
5      MOV X19, X0
(gdb) display/d $x19
1: /d $x19 = 281474976705592
(gdb) display/d $x20
2: /d $x20 = 1
(gdb) display/d $x2
3: /d $x2 = 281474976705608
(gdb) display/d $x3
4: /d $x3 = 187649984432084
(gdb) si
6      MOV X20, X1
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 281474976705608
4: /d $x3 = 187649984432084
(gdb) si
l0 () at fact_asm.s:8
8      MOV X1, #2
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 281474976705608
4: /d $x3 = 187649984432084
(gdb) si
9      CMP X19, X1
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 281474976705608
4: /d $x3 = 187649984432084
(gdb) si
10     B.GE l2
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 281474976705608
4: /d $x3 = 187649984432084
(gdb) si
l2 () at fact_asm.s:16
16     SUB X2, X19, #1
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 281474976705608
4: /d $x3 = 187649984432084
(gdb) si
17     MUL X3, X19, X20
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 4
4: /d $x3 = 187649984432084
(gdb) si
18     MOV X19, X2
1: /d $x19 = 5
2: /d $x20 = 1
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
19     MOV X20, X3
1: /d $x19 = 4
2: /d $x20 = 1
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
20     B l0
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
l0 () at fact_asm.s:8
8      MOV X1, #2
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
9      CMP X19, X1
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 4
4: /d $x3 = 5

```

```

(gdb) si
10     B.GE l2
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
l2 () at fact_asm.s:16
16     SUB X2, X19, #1
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 4
4: /d $x3 = 5
(gdb) si
17     MUL X3, X19, X20
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 3
4: /d $x3 = 5
(gdb) si
18     MOV X19, X2
1: /d $x19 = 4
2: /d $x20 = 5
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
19     MOV X20, X3
1: /d $x19 = 3
2: /d $x20 = 5
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
20     B l0
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
l0 () at fact_asm.s:8
8      MOV X1, #2
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
9      CMP X19, X1
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
10     B.GE l2
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
16     SUB X2, X19, #1
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 3
4: /d $x3 = 20
(gdb) si
17     MUL X3, X19, X20
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 2
4: /d $x3 = 20
(gdb) si
18     MOV X19, X2
1: /d $x19 = 3
2: /d $x20 = 20
3: /d $x2 = 2
4: /d $x3 = 60
(gdb) si
19     MOV X20, X3
1: /d $x19 = 2
2: /d $x20 = 20
3: /d $x2 = 2
4: /d $x3 = 2

```

```

8      MOV X1, #2
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 2
4: /d $x3 = 60
(gdb) si
9      CMP X19, X1
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 2
4: /d $x3 = 60
(gdb) si
10     B.GE l2
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 2
4: /d $x3 = 60
(gdb) si
16     SUB X2, X19, #1
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 2
4: /d $x3 = 60
(gdb) si
17     MUL X3, X19, X20
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 1
4: /d $x3 = 60
(gdb) si
18     MOV X19, X2
1: /d $x19 = 2
2: /d $x20 = 60
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
19     MOV X20, X3
1: /d $x19 = 1
2: /d $x20 = 60
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
l0 () at fact_asm.s:8
8      MOV X1, #2
1: /d $x19 = 1
2: /d $x20 = 120
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
9      CMP X19, X1
1: /d $x19 = 1
2: /d $x20 = 120
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
10     B.GE l2
1: /d $x19 = 1
2: /d $x20 = 120
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
l1 () at fact_asm.s:12
12     MOV X0, X20
1: /d $x19 = 1
2: /d $x20 = 120
3: /d $x2 = 1
4: /d $x3 = 120
(gdb) si
13     RET
1: /d $x19 = 1
2: /d $x20 = 120
3: /d $x2 = 1
4: /d $x3 = 120

```

图 4: GDB 单步调试寄存器变化过程

5 实验总结

通过本次实验，我完整地走通了中间语言 → 机器码的代码生成流程。在理论层面，我学会了如何根据 QTAC 中间语言和 Q2ARM 指令模板设计正则语言簇，并将其构建为统一的 -DFA。在实现层面，我采用表驱动的方式编写了代码生成器，体会到了“数据与逻辑分离”的工程思想。在验证层面，我在 ARM64 开发板上成功编译并运行了生成的 Fact 程序，通过 GDB 单步调试确认了每轮循环的寄存器变化与预期一致，验证了代码生成器的正确性。总而言之，我对编译器后端的工作机制有了更直观的认识。